# EGOI 2026 Editorial - Ferris Wheel

**Task Author: Nils Gustafsson**

In this task, we are given $N$ cabins numbered from 0 to $N-1$. For each cabin, we know whether the next cabin on the wheel must have a larger number ($+$) or a smaller number ($-$). We must decide whether there exists a cyclic ordering of all cabins satisfying every requirement, and if so, print one such ordering.

*Key observation*: In a valid configuration: - cabin 0 can never have a $-$ sign, because there exists no cabin with a smaller number; - cabin $N-1$ can never have a $+$ sign, because there exists no cabin with a larger number.

## Subtask 1: $N = 3$

There are only three cabins: 0, 1, and 2.

Thanks to the *key observation*, if $s_0 = -$ or $s_2 = +$, the answer is NO.

Otherwise, the answer is always YES. In this case, cabin 0 must have a $+$ sign, cabin 2 must have a $-$ sign, and cabin 1 can have either sign. If $s_1 = +$, we can print $(0, 1, 2)$; otherwise, we can print $(0, 2, 1)$.

## Subtask 2: Exactly one $+$

Again, cabin 0 cannot have a valid $-$ sign. Since there is exactly one '+' sign, the only possible valid case is that the unique $+$ sign is at position 0.

If this is not true, the answer is NO. Otherwise, one valid ordering is

$$0,\ N-1,\ N-2,\ \ldots,\ 2,\ 1.$$

In this ordering, cabin 0 is followed by a larger number, and every other cabin is followed by a smaller number. The last cabin in the ordering is 1, and it is followed cyclically by 0, which is smaller. Thus, the construction satisfies all the conditions.

## Subtask 3: Alternating signs

In this subtask, the signs alternate.

For an alternating string that admits a valid configuration, the conditions from the *key observation* ($s_0 = +$ and $s_{N-1} = -$) determine the whole string. In fact, by examining the cases: - if $N$ is odd, then $s_0 = s_{N-1}$ and the answer is clearly NO (if they are both $+$ then $N-1$ is not valid, otherwise 0 is not valid); - if

$N$ is even, then $s_0 \neq s_{N-1}$ and the answer is YES if $s_0 = +$ and NO otherwise (both $N-1$ and 0 are not valid).

In particular, in the only valid case, $N$ must be even and the string must be of the form:

$$(+, -, +, -, +, -, \ldots, +, -).$$

One valid solution is then to print the swapped adjacent pairs

$$(1, 0),\ (3, 2),\ (5, 4),\ \ldots,\ (N-3, N-4),$$

and then finish with the pair $(N-2, N-1)$. Thus the full order is

$$1,\ 0,\ 3,\ 2,\ 5,\ 4,\ \ldots,\ N-3,\ N-4,\ N-2,\ N-1.$$

Let us check the transitions. In each swapped pair $(2k+1, 2k)$, the first cabin has a $-$ sign, so going from $2k+1$ to $2k$ is valid. The second cabin has a $+$ sign, and the next printed cabin is larger, so the transition out of the pair is also valid. After the last swapped pair, cabin $N-4$ has a $+$ sign and is followed by $N-2$, which is larger. Then cabin $N-2$ has a $+$ sign and is followed by $N-1$, which is larger. Finally, cabin $N-1$ has a $-$ sign and goes back cyclically to 1, which is smaller.

This solves the alternating subtask in $\mathcal{O}(N)$ time.

## Subtask 4 and Full Solution

We now prove that the two necessary conditions from the *key observation* are also sufficient: that is, if $s_0 = +$ and $s_{N-1} = -$, then a valid Ferris wheel ordering exists.

Assume $s_0 = +$ and $s_{N-1} = -$. Construct the answer like this:

1. Print all indices $i$ with $s_i = +$ in increasing order.
2. Then print all indices $i$ with $s_i = -$ in decreasing order.

For example, if

$$s = (+, -, +, -, -),$$

then the $+$ cabins are $(0, 2)$, and the $-$ cabins in decreasing order are $(4, 3, 1)$, so we may print

$$0,\ 2,\ 4,\ 3,\ 1.$$

Let us check why this always works. Notice that there is always at least one + cabin (0) and at least one − cabin ($N-1$).

In the first part, all + cabins except the last one are followed by larger + cabins. By construction, the last + cabin is followed by $N-1$, which is strictly larger.

In the second part, all − cabins are printed in decreasing order, so each one except the last is followed by a smaller − cabin. Since we cyclically return to the first printed cabin at the end, the last − cabin is followed by 0, which is strictly smaller.

This proves both the construction and the characterization of the impossible cases.

For Subtask 4, we have $N \leq 1000$, so it suffices to maintain one list while iterating through the cabins and inserting each index either at the beginning or at the end. Since inserting at the beginning takes linear time, this implementation runs in $\mathcal{O}(N^2)$.

For the full solution, we need a faster implementation. One possibility is to store all the + cabins into one list, in increasing order, and all − cabins into another list, in decreasing order. Then print the + list followed by the − list. This can be done in $\mathcal{O}(N)$ time, by first reading the string and storing the + cabins and then reading it backwards and storing the − cabins.

## Alternative full solution

There is also a different approach, built on Subtask 3. Every string can be seen as an alternating sequence of blocks of consecutive + or −.

For example, if

$$s = (+,+,+,-,-,+,-,+,+,-,-),$$

we split it into blocks as follows

$$(+,+,+),(-,-),(+),(-),(+,+),(-,-)$$

and thus group the corresponding cabins like this:

$$(0,1,2),(3,4),(5),(6),(7,8),(9,10).$$

We know how to deal with alternating strings, and we can apply the ordering of Subtask 3 to the blocks. It only remains to sort the cabins inside each block.

- if the block is made of [+], we place the cabins inside it in increasing order;
- if the block is made of [−], we place the cabins inside it in decreasing order.

In the example above, first order the numbers according to the blocks in this way

$$(0, 1, 2),\ (4, 3),\ (5),\ (6),\ (7, 8),\ (10, 9).$$

and then applying the construction of Subtask 3 to the alternating blocks gives:

$$(4, 3),\ (0, 1, 2),\ (6),\ (5),\ (7, 8),\ (10, 9).$$

This gives us the following construction:

$$4,\ 3,\ 0,\ 1,\ 2,\ 6,\ 5,\ 7,\ 8,\ 10,\ 9.$$

It can be checked, similarly to the other constructions, that this always works. This construction can also be done in $\mathcal{O}(N)$ time.

# EGOI 2026 Editorial - Ovenmasters

**Task Author: Tomohisa Hachiya, Yuki Tanaka**

Let us call the pizza currently lying on a table the *top* pizza of that table. After the first $M$ arrivals all tables are occupied. From then on, a baker with rank $x$ chooses the table whose top pizza has the largest rank smaller than $x$. If no top pizza has rank smaller than $x$, the baker leaves and does not appear in any tray stack.

The first useful observation is about a single table. Whenever a baker replaces a pizza on a table, her rank must be larger than the rank of the pizza she ate. Therefore, in every stack, the ranks have to be increasing from bottom to top. If this is not true, the answer is immediately `NO`.

The first $M$ arrivals are also fixed: they are the bottom elements of the $M$ stacks, in table order. After this, the actual table labels no longer matter. The only relevant information is the sorted order of the current top pizzas.

## Subtask: $M = 1$

With one table, the process is very simple. The stack must be increasing. Any baker not in the stack has to leave, which is possible only after the current top pizza has become larger than that baker's rank.

Thus we can start with the first stack value and then repeatedly take the smallest rank that is currently possible:

- either the next value in the stack, which replaces the current top;
- or a missing value smaller than the current top, which leaves.

If at the end some missing value is larger than the final top pizza, it could never have left, so the instance is impossible.

## Subtask: $M = 2$, $N \leq 200$, and no baker leaves

In this subtask every baker appears in one of the two stacks. After the first two arrivals, the next arrival must therefore be the next unused value of one of the two stacks.

Let table 0 be the table with the smaller bottom value, and table 1 the table with the larger bottom value. Their current top values are always $c_0 < c_1$.

Now consider a possible next arrival with rank $x$.

- If $c_0 < x < c_1$, then table 0 contains a pizza better than $x$, while table 1 does not. So $x$ must go to table 0.

- If $x > c_1$, then both tables contain a pizza better than $x$, and the worse of those two pizzas is $c_1$. So $x$ must go to table 1.
- If $x < c_0$, then there is no better pizza for $x$. This case cannot happen in this subtask, because no baker leaves.

Therefore, to decide whether the next unused value of a stack can be taken now, we just check whether it falls into the interval for that stack.

At each step there are at most two candidates: the next unused value of the first stack and the next unused value of the second stack. We test which of them are currently possible and append the smaller possible one to the answer. If neither candidate is possible, the answer is `NO`.

This greedy choice gives the lexicographically smallest order, because every valid order with the current prefix must choose one of these possible candidates next. The smaller one is therefore always best.

The implementation can be completely direct and runs in $O(N)$ time.

## Keeping the tables sorted

Suppose the current top ranks of the tables, sorted increasingly, are

$$c_0 < c_1 < \cdots < c_{M-1}.$$

A baker with rank $x$ will choose table $i$ exactly when

$$c_i < x < c_{i+1},$$

where for the last table there is no upper bound. This is because $c_i$ is then the worst available pizza that is still better than $x$.

In particular, if table $i$ is chosen, its top changes from $c_i$ to some value $x < c_{i+1}$. Thus the sorted order of the tables does not change. This lets us sort the tables once by their initial top pizza and keep them in that order for the whole solution.

It is also convenient to add one fake table for the bakers who leave. Its current value is $-1$, and it is placed before all real tables. A baker who does not appear in any stack can be handled as belonging to this fake table: such a baker may arrive exactly when

$$-1 < x < c_0,$$

which means that no real table has a better pizza for her.

### Subtask: $N \leq 200$ and no baker leaves

For more than two tables, the same idea still works. Every baker appears in some stack, so the next arrival must be the first unused value of one of the stacks.

We keep the current top value of every table. Repeatedly look at the next unused value of each stack, check whether that value would currently choose this table, and append the smallest value that passes the check. If no stack has a possible next value, the answer is `NO`.

The lexicographic argument is the same as above: after a fixed prefix, we choose the smallest value that can legally be the next arrival. A straightforward implementation is fast enough for $N \leq 200$, for example in $O(N^2M)$ time.

### Subtask: $M \leq 10$

For larger $N$ but small $M$, we should avoid scanning all unused ranks after each arrival. Instead, we will scan the tables.

First sort the real tables by their first stack value. We also add the fake table for bakers who leave. After sorting, think of these as rows $0, 1, \ldots, M$, where row 0 is the fake table and the remaining rows are the real tables.

Each row $i$ stores two things:

- a current value $c_i$, initially its first value ($-1$ for the fake row);
- a list of remaining values that still have to be processed in increasing order.

For a real row, the remaining values are the rest of its stack. For the fake row, they are all ranks that do not appear in any stack, sorted increasingly.

The values $c_i$ are always sorted:

$$c_0 < c_1 < \cdots < c_M.$$

For row $i$, let $u_i$ be the first value in its remaining list. This is the only value from row $i$ that can be processed next, because values in one stack have to appear in their stack order.

When can we process $u_i$? The answer is exactly when it would currently choose row $i$. This means

$$c_i < u_i < c_{i+1},$$

with no upper bound when $i = M$. For the fake row, this says $c_0 < u_0 < c_1$, which is exactly the condition that $u_0$ has no better pizza available and leaves.

Now scan the rows from left to right and find the first row whose $u_i$ is currently possible. This gives the smallest possible next arrival. Indeed, if row $i$ is possible,

then $u_i < c_{i+1}$. But every value in a later row is larger than its current value, which is at least $c_{i+1}$. So no later row can give a smaller next rank.

After we find this row, append $u_i$ to the answer, set $c_i = u_i$, and remove $u_i$ from the row. Then start another scan from row 0. If no row is possible but some values are still unprocessed, the answer is `NO`.

This takes $O(NM)$ time after sorting, which is easily fast enough for $M \leq 10$.

## Full solution

The full solution is the same row-based algorithm. The only extra care needed is to avoid starting the scan again from row 0 after every processed value.

Build $M + 1$ rows:

- one fake row containing $-1$ followed by all ranks that do not appear in the input;
- one row for each table stack.

Before sorting the rows, save the first $M$ output values: they are the bottom elements of the original table stacks, in the original table order.

Then sort all rows by their first value. The fake row, whose first value is $-1$, will become the first row. For each row, remove its first value and store it as $c_i$. The remaining values in the row are processed from smallest to largest.

Now maintain an index $r$, initially 0.

- If the current row has no remaining values, move to the next row.
- Otherwise let $x$ be the first remaining value in this row.
- If $x < c_r$, the row is not increasing, so output `NO`.
- If this is the last row, or $x < c_{r+1}$, then $x$ is the smallest possible next arrival. Append it to the answer, set $c_r = x$, remove it from the row, and move one row left if possible.
- Otherwise, $x$ is still too large: it would currently choose a later table. Move one row right.

If the scan finishes and the answer contains all $N$ ranks, output `YES` and the answer. Otherwise output `NO`.

### Correctness proof

We prove that the algorithm outputs `NO` exactly for impossible inputs, and otherwise outputs the lexicographically smallest valid order.

First, each real stack has to be increasing. Every time a baker appears in a stack, she replaced a better pizza, hence her rank is larger than the previous top of that table. Therefore a non-increasing stack is impossible, and the algorithm correctly rejects it.

Second, once the real tables are sorted by their current top ranks, their order never changes. If a baker with rank $x$ chooses row $i$, then $c_i < x < c_{i+1}$, or row $i$ is the last row. After the replacement, the new value of row $i$ is still before row $i+1$ in sorted order.

Third, the fake row exactly represents bakers who leave. A missing rank $x$ can leave precisely when $x$ is smaller than the smallest real current top. This is the same condition as being processable in the fake row.

Now consider any moment during the algorithm. In row $i$, the first unprocessed value is the only value from that row that can possibly be processed next, because stack order must be preserved. It is possible exactly when it lies between $c_i$ and $c_{i+1}$, with no upper bound for the last row.

Among all possible next values, the smallest one is in the leftmost possible row: every possible value in row $i$ is smaller than $c_{i+1}$, while every value in a later row is larger than its own current value, and therefore at least $c_{i+1}$.

Thus, whenever the algorithm processes a value, it appends the smallest rank that can appear next in any valid continuation. This is exactly the greedy choice needed for lexicographic minimality. If the algorithm cannot process any remaining value, then no valid next arrival exists, so no valid order exists.

Therefore, if the algorithm outputs `YES`, the produced order is valid and lexicographically smallest; if it outputs `NO`, no valid order exists.

### Complexity

Sorting the rows and the missing ranks takes $O(N \log N)$ time. The scan over the rows is linear in the number of processed values plus the number of row moves, so it is $O(N + M)$ after sorting.

The memory usage is $O(N)$.

# EGOI 2026 Editorial - Biscuits

**Task Author: Nemanja Majski**

In this task, we are given a stack of $N$ biscuits with known weights. Aurora first chooses an integer $X \geq 0$. Bianca then chooses an integer $Y \geq 0$ such that $Y \neq X$ and at least $Y$ biscuits are still in the stack. Aurora eats the top $Y$ biscuits. If any biscuits remain after that, Bianca eats the next one.

Both players play optimally to maximize the total weight of biscuits they eat. Our task is to find the total weight of biscuits that Bianca can eat for the initial stack, and also for the stack obtained after each of the $Q$ updates, where one biscuit is replaced by another biscuit with a possibly different weight.

## Subtask 1: $Q = 0$ and all $W_i = 1$

When every biscuit weighs the same, both players only care about how many biscuits they eat.

In this case, Bianca will always choose $Y$ as small as possible, to leave as little biscuits as possible to Aurora. Therefore, Aurora will choose $X = 0$ in each turn, to guarantee that she will get to eat at least one biscuit, and then Bianca will choose $Y = 1$ to guarantee that Aurora gets to eat just one biscuit.

Therefore Bianca gets to eat every second biscuit, starting with the second topmost biscuit on stack - summing this together, she gets to eat biscuits with total weight of

$$\left\lfloor \frac{N}{2} \right\rfloor.$$

Note that there are no updates in this subtask, so we just print this value once and we are done.

## Subtask 2: $N \leq 3, Q \leq 5$

For very small $N$, we can try all possible choices of $X$ and $Y$ at each step using bruteforce.

We make a function that is given an array of weights of biscuits in a stack and computes the optimal score of the game recursively.

When trying to find the score that Bianca will be able to obtain in this situation, we can simply try all possible $X$ that Aurora can choose, then all possible $Y$ that Bianca can choose, such that $Y \neq X$. We can then recursively find the

optimal score with which the game will be finished after the performed turn. To get the value of our turn, we add $W_Y$ to previously calculated score.

Bianca will then choose the $Y$ that maximizes this score, and Aurora will choose $X$ that minimizes it, among all possible choices of $Y$.

There are at most four possible values of $X$ and $Y$, so this is fast enough for $N \leq 3$ and $Q \leq 5$.

## Subtask 3: Weights are non-increasing

More formally, in this subtask, we are guaranteed that at any point

$$W_0 \geq W_1 \geq \cdots \geq W_{N-1}.$$

Claim: Bianca can guarantee her score to be at least $W_1 + W_3 + W_5 + ... + W_{N-1-(N \bmod 2)}$.

We will prove this. We start by demonstrating a working strategy for Bianca to achieve at least as high score for all $N$ and proving it by induction.

For $N = 1$, the claim is trivial as Bianca can guarantee her score to be non negative.

Now assume that the claim (score of Bianca can be at least the sum of weights of biscuits on the even position with respect to the top of the stack) was proven for stacks of size $M = 1...N-1$. When playing on a stack of biscuits of size $N$ with the weights $W_0, W_1, ..., W_{N-1}$, Bianca will do the following strategy: If Aurora chooses $X = 1$, then Bianca chooses $Y = 0$ and eats the biscuits of weights at least $W_0 + (W_2 + W_4 + ...) \geq W_1 + W_3 + W_5 + ...$, which proves the claim. Otherwise, Bianca chooses $Y = 1$ and eats the biscuits of weights at least $W_1 + (W_3 + W_5 + ...)$, once again verifying the claim.

On the other hand, Aurora can ensure Bianca eats biscuits with total weight at most $W_1 + W_3 + W_5 + ....$ She can achive that by always choosing $X = 0$. That guarantees that in $K$th turn (we number turns starting from zero) Bianca eats a biscuit of weight at most $W_{2K+1}$, as Aurora herself eats the biggest remaining biscuit in each turn.

Therefore the score we need to find is just the sum of the biscuits on the even positions with respect to the top of the stack.

We can easily find this sum in $O(N)$ for the initial stack, and update it after each of $Q$ updates in $O(1)$, leading to time complexity $O(N + Q)$.

## Subtask 4: $N, Q \leq 100$

We can notice that the final score of Bianca after some turns depends only on the sum of the weights of all the biscuts she previously ate, and on the weights of the biscuits that remain in the stack (they form a suffix on the original array).

We can see the repetitive subproblem structure and therefore it is natural to try to introduce dynamic programming.

Define $dp[i]$ as the score Bianca will get if she and Aurora played the game on the stack of cookies with weights $W_i, W_{i+1}, \ldots W_{N-1}$.

Additionally, we will define $dp[N] = 0$, because playing the game on the empty array results in the score of zero.

We also know that $dp[N-1] = 0$, as Aurora can chose $X = 0$, forcing Bianca to chose a bigger value of $Y$ and getting to eat the cookie at index $N-1$.

Now we will start computing the values of $dp$ in decreasing order of indices.

Assume we have already computed $dp[i+1], dp[i+2], \ldots, dp[N]$ and are now computing $dp[i]$.

We now want to iterate on the possible values of $X$ and $Y$ to find the answer using the precomputed $dp$ values.

First, assume that Aurora has already picked the value for $X$. Let us try to find the value $Y$ that Bianca should pick.

When Bianca picks a particular $Y$, she will eat the biscuit at position $i+Y$, and then the game will continue on stack of biscuits with weights $W_{i+Y+1}, W_{i+Y+2}, \ldots, W_{N-1}$. So her score will be $W_{i+Y} + dp[i+Y+1]$.

Among all the values of $Y$ Bianca can pick, she will pick the one that maximizes the value of $W_{i+Y} + dp[i+Y+1]$, under the constraint $0 \leq Y \leq N-i-1$ and $Y \neq X$.

Define a function $f(X)$ as the score Bianca gets under optimal gameplay, if Aurora picks this $X$. We know that

$$f(X) = \max_{0 \leq Y \leq N-i-1, Y \neq X} W_{i+Y} + dp[i+Y+1].$$

Among all possible choices for $X$, Aurora will pick the one that minimizes $f(X)$.

More formally, the $dp[i]$ can be calculated as

$$dp[i] = \min_X(\max_{Y \neq X}(W_{i+Y} + dp[i+Y+1])).$$

In each iteration,we can compute $dp[i]$ by iterating over all the possible values of $X$ that Aurora can pick and compute $f(X)$ by iterating over all the possible values $Y$ that Bianca can pick and finding the largest value of $W_{i+Y} + dp[i+Y+1]$ under the constratints. Then the smallest of the $f(X)$ values will be the value of $dp[i]$. For a single stack, this takes $O(N^3)$ time, and we will compute this from scratch for each update, so the total time complexity is $O(N^3Q)$, which is fast enough for $N, Q \leq 100$.

### Subtask 5: $N \leq 10,000$, $Q \leq 100$

We will optimize the solution of the previous subtask. The number of states is small (there are $N$ values of $dp[i]$, so let us focus on optimizing the process of computation of $dp[i]$.

Intuitively, if Bianca picks a large value $Y$, it will cause her to give up many biscuits to Aurora, which makes some sense if she is trying to get to a very big biscuit, but since the weights of the biscuits are pretty small, it does not make sense for her to give up too many biscuits by picking a very large $Y$.

Take the following example. Stack consists of 200 biscuits of weight 1, followed by a single biscuit of weight 50. Should Bianca set $Y$ such that she takes the biscuit of weight 50?

If she does, her score will be 50. But if she instead always chooses $Y$ as small as possible, she will end up eating at least half of the biscuits (it is the same strategy as in subtask 1 but applied to a general array) and then her score will be at least 100, which is much better.

Notice that this would not hold if the weight of the big biscuit would be, say 1000. The reason why this is happening is that the weights of the biscuits are pretty small. Let $M$ be the maximum weight a biscuit can have.

If we could somehow prove that Bianca will always pick a small value $Y$, the number of possible $f(X)$ and corresponding $Y$ we need to iterate through would decrease and therefore the dynamic programming would be faster.

More precisely, let us assume that biggest value of $Y$ Bianka might pick in such scenario is $Y_{max}$. Then note that if Aurora were to pick any $X$ such that $X > Y_{max}$, it is the same as if she allowed Bianca to pick any number, as Bianca would have not picked $X$ anyway. Therefore there is always an optimal strategy where Aurora picks $X$ such that $X \leq Y_{max}$.

Therefore, if we can somehow get a good bound on $Y_{max}$, we can reduce the number of $X$ values we check.

Suppose we are computing $dp[i]$ and we are wondering what is the biggest index $j$ of a biscuit Bianca might choose to eat next, by picking $Y = j - i$.

If there would be $j > i + 2 \cdot M$ such that $Y = j - i$ is the optimal choice for Bianca on this suffix, then her score in this case would be $W_j + dp[j + 1]$. But she can do another strategy (somewhat similar to subtask 1 and the example above): while she can pick $Y \neq X$ such that the remaining suffix will be of length at least $N - j + 1$, she will pick the smallest possible $Y$ (so if $X = 0$, then $Y = 1$, otherwise $Y = 0$) and keep playing the game, collecting at least half rounded down of the biscuits in $i...j - 2$, in other words, eating biscuits with total weight at least $\lfloor \frac{j-i-1}{2} \rfloor$.

Once this is no longer possible, in response to Aurora's choice of $X$, Bianca will choose $Y$ such that she either gets to eat the biscuit at position $j$ or $j - 1$

in this turn. If she can choose the former, then with this strategy she clearly achieves better score using $Y = 0$ or $Y = 1$ than if she originally chose $Y = j - i$, which is a contradiction. Otherwise, her score with this strategy is at least $\lfloor \frac{j-i-1}{2} \rfloor + W_{j-1} + dp[j]$.

However we can prove that $dp[i]$ is non increasing (for a suffix starting with $i$, Bianca can always copy her strategy for suffix of length $i+1$ and pretend the $i$th biscuit does not exist and get at least as good score as $dp[i+1]$), therefore the value above is at least $\lfloor \frac{j-i-1}{2} \rfloor + W_{j-1} + dp[j+1] \geq M + 1 + dp[j+1] > W[j] + dp[j+1]$.

which is a contradiction once again, proving that the optimal $Y$ is always at most $2 \cdot M$. This gives us an upper bound of $Y_{max} \leq 2 \cdot M$.

This gives us a method to compute the transitions in $O(M^2)$. We can further optimize this by noticing that computing $f(X)$ and $f(X+1)$ is very similar and reusing the information computed cleverly. For example, we can do the following - we will introduce two new arrays, $pref$ and $suf$. We define $pref[j]$ as

$$pref[j] = \max_{i+1 \leq k \leq j}(W[k+1] + dp[k+2]).$$

Similarly, we define $suf[j]$ as

$$suf[j] = \max_{j \leq k \leq i+Y_{max}}(W[k+1] + dp[k+2]).$$

We can compute these arrays in $O(M)$ just before computing $f(X)$ values for $dp[i]$. Then for any $X$, we can find $f(X) = \max(perf[X-1], suf[X+1])$ in O(1) time.

With this optimization the time complexity is $O(N^2Q)$, which is still not enough to pass this subtask.

Then the number of transitions in the dynamic programming for previous subtask goes down to $O(M)$ and the overall complexity can be brought down to $O(NQM)$.

## Subtask 6: $N, Q \leq 10,000$

There is also another way to make the dynamic programming from subtask 4 faster.

Consider the stack of biscuits of weights $W_i, ..., W_{N-1}$. Depending on which $Y$ Bianca picks, the score will be one of the values $W_j + dp[j+1]$ for $i \leq j \leq N$. To account for the possiblity of Bianca not eating any biscuit, we define additionally $W_N = 0$ and $dp[N+1] = 0$.

Aurora can block at most one of these possibilities by choosing proper $X$ and then Bianca will pick the highest scoring remaining option. We can formalize this as

$$dp[i] = \text{secondmax}_{i \leq j \leq N}(W_j + dp[j+1])$$

Here secondmax($S$) is the second biggest element of multiset $S$ (or 0 if the multiset has less than two elements).

Using this formula, we can compute $dp[i]$ in $O(N)$. But we need to do it faster! To do so, notice that when moving from suffix $i+1$ to suffix $i$, the multisets passed to the secondmax stays the same, except one new possibility gets added, corresponding to Bianca choosing $i$, resulting in the score $W_i + dp[i+1]$. But recalculating secondmax for a multiset with one more new value is actually easy and can be done in $O(1)$ - we just need to mantain the two largest values in the multiset and update them with the new value.

This gives an $O(N)$ computation of $dp$ for the original array and after each update, leading to a total time complexity of $O(NQ)$ which is fast enough for $N, Q \leq 10,000$ with a proper implementation.

## Full solution

We will explore the structure of the $O(N)$ per stack dynamic programming to be able to efficiently process updates. First of all, define $W_N = 0$ in addition to $dp_N = 0$ so that we can state

$$dp[i] = \text{secondmax}(W_j + dp[j+1] \mid i \leq j \leq N).$$

without the 0, because that is just the $j = N$ case.

The most complex part of the formula for the computation of the dynamic programming is the secondmax formula, so we will focus on simplifying it. To achieve this, we start by introducing a new dynamic programming $t[i]$ such that $t[i] = W_i + dp[i+1]$. Then we have that

$$t[i] = dp[i+1] + W_i = \text{secondmax}(W_j + dp[j+1] \mid i+1 \leq j \leq N) + W_i$$

which we finally rewrite using the $b[i]$ definition as

$$t[i] = \text{secondmax}(t[j] \mid i+1 \leq j \leq N) + W_i$$

which is simpler since now we have the single $dp$ (actually $t$) values in the secondmax function, not the sums.

But wait, we need $dp[0]$, how do we get that? We will do a little trick and say that we have $W_{-1} = 0$ and extend the computation to obtain $b[-1]$ as well:

$$t[-1] = \text{secondmax}(t[j] \mid 0 \leq j \leq N) + W_{-1} = dp[0]$$

Notice that when we are computing value of $t[i]$, we do not need the full information about the values $t[i+1], \ldots, t_{N-1}$, but rather, we are interested in

two special values, let us call them $A[i+1] = \max\{t[i+1], t[i+2], \dots t[N]\}$ and $B[i+1] = \text{secondmax}\{t[i+1], t[i+2], \dots t[N]\}$, where clearly $A[i+1] \geq B[i+1]$. These are closely related to $t[i]$ by $t[i] = B[i+1] + W_i$.

Using $A[i+1]$ and $B[i+1]$ we are now interested in computing $A[i]$ and $B[i]$. They are the two greatest values from the multiset $\{t[i], t[i+1], t[i+2], \dots t[N]\}$ which is the same as the two highest values in the multiset $\{B[i+1]+W_i, A[i], B[i]\}$, but since $B[i] \leq B[i+1]+W_i$ and $B[i] \leq A[i]$, these are just $A[i]$ and $B[i+1]+W[i]$ (possibly swapped, we do not know which one is bigger).

Well this does not seem too helpful so far - we are now keeping track of two values ($A$ and $B$) instead of one (the dynamic programming). Let us try to simplify this to be able to keep track of just one value. As a thought experiment, what would the values of $A[0]$ and $B[0]$ be if at the values $A[N], B[N]$ were not both 0, but both 100. Let us denote such arrays as $A'$ and $B'$. We can notice that $A'[i] = A[i] + 100$ and $B'[i] = B[i] + 100$ will hold not just for $i = N$, but also for the rest of the indices. Notice that whenever $B[i] + W_i > A[i]$, it will also hold that $B'[i] + W_i > A'[i]$

Inspired by that observation, instead of keeping track of the values of $A[i]$ and $B[i]$, we will keep track of the values $A[i] + B[i]$ and $A[i] - B[i]$. We will see that value of both can be computed separately. Computing final value of $A[i] + B[i]$ is easy: from the nature of updating $(A[i], B[i])$ using $(A[i+1], B[i+1])$ we can see that $A[i] + B[i] = A[i+1] + B[i+1] + W[i]$, therefore $A[i] + B[i] = \sum_{j=i}^{N-1} W_j$.

Meanwhile, the value of $A[i] - B[i]$ can be computed in a for loop. We will create a new array of values $D[i] = |A[i] - B[i]|$. Even with the absolute value, this still allows us to reconstruct $A[i]$ and $B[i]$ uniquely using the difference and sum, because we know that $A[i] \geq B[i]$. We start by setting $D[N] = 0$ and then iterate through the array from index $N-1$ to index 0. When we are at index $i$, we can see that

$$D[i] = |A[i] - B[i]| = |A[i+1] - B[i+1] - W[i]| = |D[i+1] - W[i]|$$

so we can calculate $D[i]$ values without explictly calculating $A, B$ or anything else.

However, this computation is still $O(N)$ per query, so we need something faster.

Updates are local, but they force us to recompute the whole prefix of the array $D[i]$, starting with the point of change and going backwards all the way to 0.

What if all updates happen after, let us say, index $K$? We would like to maintain some sort of prefix structure that will let us handle updates without having to iterate over the first $K$ indices each time. Notice that value of $D[i]$ depends only on value of $D[i+1]$ and $W_i$. We can extend this observation - the value $D[i]$ depends only on the values $W_i, W_{i+1}$ and $D[i+2]$ and so on. And so we can see that, after each update, the new value of $D[0]$ can be determined from the new

value of $D[K]$ and the values $W_i$ for $0 \leq i \leq K$, but the only thing that changes is $D[K]$.

Notice the constraint $W_i \leq 50$: it also implies that $D[i] \leq M$ (we can prove this by induction, as $D[i] \leq M$). Therefore, $D[K]$ can take at most $M + 1$ different values.

The idea is to precompute value of $D[0]$ for all possible values of $D[K]$. As there are at most $M + 1$ possible values $D[K]$ can take, this can be done in $O(KM)$. Then, when we need to compute the difference after some update, we can compute $D[K]$ and then look up the corresponding precomputed value to find $D[0]$.

But in the original problem, the updates can happen anywhere in the array and therefore using a simple prefix sum like this is not sufficient. Instead, we will use a segment tree based on a similar idea.

We define function $f_{L,R}(x)$ (for $0 \leq L < R \leq N - 1$) as follows. Assuming that value $D[R + 1] = x$, what will be the value of $D[L]$? If for some $j$ such that $L \leq j \leq R$ we know values of $f_{L,j}$ and $f_{j,R}$ for all $x \in [0, ..., M]$, we can calculate $f_{L,R}$ for all $x \in [0, ..., M]$ by observing that $f_{L,R}(x) = f_{L,j}(f_{j,R}(x))$.

When we build a segment tree over the array, we will have the node responsible for interval $[L, R]$ maintain the value of function $f_{L,R}$ for all $x \in [0, ..., M]$.

Computing the leaf nodes, $f_{L,L}$, is easy when we know $W_L$: we simply get $f_{L,L}(x) = |x - W_L|$ and we can do this in $O(M)$ whenever needed.

For the other nodes, we can compute $f_{L,R}$ using the precomputed $f$ for the two children - this can be simply done using the formula above, in $O(M)$.

To find the optimal score, we need to maintain $S = \sum_{i=0}^{N-1} W_i$ and find $D[0] = f_{0,N-1}(0)$. Then we calculate $B[0] = \frac{S-D[0]}{2}$ and observe that $dp[0] = t[-1] = B[0]$, so we will print $\frac{S-D[0]}{2}$ in the beginning and after processing each update.

This segment tree can be set up in $O(N \cdot M)$ time. Every update can be processed in $O(M \log N)$ time and the score can be foud in $O(1)$ time. The total time complexity will be $O(N \cdot M + Q \cdot M \cdot \log N)$, which is fast enough for 100 points.

# EGOI 2026 Editorial - Census

**Task Author: Simon Lindholm, Joakim Blikstad**

In this somewhat unusual problem, $N \leq 100$ instances of your program run at once, need to figure out the value of $N$. Each instance has a distinct ID $I \in \{0, \ldots, M-1\}$ with $M \leq 10^5$. There are $10^{18}$ locations (memory cells) that can be used for communication, all of which initially have the value 0. In each round, each still-active instance either *reads* from a location $P \in [0, 10^{18})$ or *writes* a value $V$ to one. Reads happen before writes, and no two instances may write to the same location in the same round. Each instance must eventually output $N$. We aim to minimize the number of rounds used. For full score we must use at most 61 rounds, and all written values $V$ must be either 0 or 1.

## Subtask 1: consecutive IDs, $M \leq 100$ (11 points)

A natural start in this subtask is to have each instance write a 1 to the position given by its ID. Then $N$ will equal the number of ones in the range $0, \ldots, M-1$, or equivalently, the position of the leftmost location with the value 0.

For 8 points, you can count the number of ones using a loop from 0 from $M-1$. For the full 11 points, you can use a binary search for the boundary.

## Subtask 2: $N \leq 2$ (12 points)

With non-consecutive IDs and $M$ up to $10^5$, both solution ideas from subtask 1 fail.

One thing you can do to revive the idea of a counting loop is to use randomization to reduce the value of $M$. Every instance discards its $I$ value, and generates a new random one from scratch in the range $0 \ldots 60$ (e.g. using `random_device()() % 61` in C++, or `random.randrange(61)` in Python). With high probability these new $I$ values will still be distinct. The instances can then write a 1 to position $I$, and loop from 0 to 60 to count the number of written values. Because of randomness, this solution may get judged as Incorrect because of multiple writes to the same position. (The judge test data had about 100 test cases with two instances, so this occurs with probability $1 - (60/61)^{100} \approx 0.8$.) Thus, it may require a few resubmits to get it to pass.

Another thing you can do which generalizes better to the full problem is to use a tree-based construction. We start by having each instance write a 1 to its own instance ID, and then pair up IDs 0 and 1, 2 and 3, 4 and 5, and so on. Each instance reads the value at its paired ID. If this value is 1, it must be the case that $N = 2$, and we can exit early. Otherwise, it can change its ID to the

lower of the two IDs in the pair. Since only one of the IDs corresponded to an instance, this can not result in an ID collision.

We can now repeat this with IDs being paired up as 0-2, 4-6, 8-10, ..., then 0-4, 8-12, ..., then 0-8, ..., etc., until we reach the point where ID 0 would be paired up with an ID $\geq M$. At that point, we can be sure that only one instance is running, and output $N = 1$ and exit.

## Subtask 3: $M \leq 8000$, large writes allowed ($V \leq 10^9$) (22 points)

In this subtask, it is natural to reuse the tree-based construction from the previous subtask. However, now instead of exiting early we must compute the full count of program instances. Instead of writing 1s, let us write the count of instances in the current subtree.

We again start by having each instance write a 1 to its own instance ID, pair up IDs 0-1, 2-3, etc., and have each instance read the value at its paired ID. Now in principle, we want to add our own sum and the paired one, and write the total to the lower of the IDs. If both sums are non-zero, however, this may cause a double write, which is invalid. Thus, in this case, we make sure that only the instance with the lower of the two IDs performs the write, and the other one is transitioned into a "dead" state where it does not perform any further writes. In this state, in the rounds when other instances would write, it instead performs a dummy read, ignoring the response.

Then, we continue by pairing 0-2, 4-6, ..., then 0-4, ..., etc., and in the end we will be left with the sum written in location 0.

## Subtask 4: full problem (up to 55 points)

For the full problem, there are multiple approaches one can use, scoring various amounts of points. Here we outline a couple of them.

**Binary adder:** $O(\log M \log N)$

One idea is to make use of the same construction as in subtask 3, but writing numbers in binary, using multiple locations instead of just one. All numbers written will be at most $N \leq 100$, and therefore only need $\lceil \log(N+1) \rceil = 7$ bits of memory. We can split each round in $7 + 7$ subrounds, with the first 7 spent reading the paired ID's value, and the second 7 spent writing the sum. The number of rounds used will be $2 \times 7 \times \lceil \log M \rceil + 1 = 239$.

This number can be optimized slightly. One basic idea is to use $1, 2, \ldots, 6$-bit numbers for the first 6 summations, instead of 7-bit ones. This reduces the number of rounds to 203.

Another idea is optimize writes by making use of dead instances. Each number that counts the number of instances in a subtree will, when written in binary,

require a number of bits which is less than or equal to the number of instances in that subtree. Thus, we can parallelize the write of the sum by distributing the responsibility of writing each binary digit to separate instances. Over the course of the tree summation process, each instance is able to learn the number of nodes within their subtree with an ID lower than their own, and can use this as an index into the binary number and know which digit to write. (Similar to before, some instances will not required to write a digit and need to perform a dummy read instead.) We get a number of needed rounds of $(1+7) \times \lceil \log M \rceil + 1 = 137$, or 116 if combined with the previous idea.

It is tempting to try to also optimize reads in a similar fashion. Unfortunately, this quickly gets tricky. Output bits for the addition depend on *all* earlier input bits, and even if we parallelize reading input bits, we somehow need to combine the knowledge of all these 7 bits to compute one output bit. This takes at least $\lceil \log 7 \rceil$ rounds. It may be possible to use a construction like a Kogge-Stone adder, which would give a solution with $O(\log M \log \log N)$ rounds, but the constant factor would probably not be an improvement.

A fourth idea is to use randomness. We could use the same kind of regeneration of $I$ as in subtask 2 to reduce $M$ slightly; however, we can only reduce $M$ to around $N^2$ before we start getting collision (based on the Birthday Problem). Since $N = 100$ and $M = 100\,000$, this only helps very little, even with many resubmits. However, we can make use of a slightly more advanced version of this idea. Start by mapping $I$ to another value in the same range $0, \ldots, M-1$ using a random permutation (e.g. multiply by a randomly chosen constant mod $M$). Then, we probably do not need $1, 2, \ldots, 6, 7, \ldots,$ 7-bit number for the summations, but can almost certainly get away with something like $1, 2, \ldots, 2, 3, 4, 5, 6, 7, 7$. Asymptotically, this gives a $O(\log^2 N + \log M)$ solution, which almost (but not quite) achieves full score.

**Unary adder:** $O(\log M \log N)$

Instead of using binary numbers, the previous solution can also be implemented using unary numbers, i.e. with streaks of consecutive 1s being written to indicate the count of number of instances. While this sounds slower, it actually uses exactly the same number of rounds: reading a number can be done using a binary search, and writing can be done in one round by parallelizing the write as described above.

One way of viewing this solution is that after each instance writes a 1 to the location corresponding to its ID, we perform a merge sort of the values at locations $0, \ldots, M-1$.

**Sorting networks:** $O(\log M)$

Instead of using a merge sort, we can do sorting using a sorting network. Sorting networks are based on the idea of performing multiple parallel rounds of sorting, where in each round the items to be sorted are partitioned into disjoint pairs,

and each pair $(a, b)$ gets sorted in-place, by swapping $a$ and $b$ if $a > b$. We can simulate a sorting network in our setup by having each instance be responsible for a single 1, and when a pair of locations $i$, $j$ (with $i < j$) is to be sorted, the instance responsible for for location $j$ reads the value at location $i$, and if it is zero, writes 0 to location $j$, 1 to location $i$, and shifts its responsibility to location $i$. In other cases, instances perform dummy reads, to ensure they use exactly three queries per round of the sorting network.

At the end, each instance performs a binary search to count the number of 1 in the sorted array, which will all be placed consecutively at one end.

The efficiency of this depends on the depth (i.e. round count) of the sorting network. With a bitonic sorting network, we get a solution that uses $O(\log^2 M)$ rounds (in practice around 450). More interestingly, there are results in theoretical computer science showing the existence of sorting networks that use only $O(\log M)$ rounds. In practice, however, these have terrible constants: the best published value lies around $1800 \log M$ or so, making this solution impractical.

**Lazy ternary adder:** $O(\log M)$

The main issue that prevents the binary adder solution from reaching full score is that while writing can be parallelized to take $O(1)$ rounds at each level, reading can not. In an ideal world, we would have one instance responsible for each digit of the binary number saved for each subtree. That instance would read the corresponding digit of the other subtree, compute the digit in that location for the sum, and either write that back or shift its responsibility to the digit one step above. The reason this does not work is carries: in the summation of two binary numbers, a carry from the least significant bit sum can ripple all the way to the most significant bit.

While there are ways to optimize latency of binary addition slightly (see e.g. the mention of Kogge-Stone adder above), one way to get down to $O(\log M)$ is to switch to a ternary (base 3) number representation, and make it *lazy*, with digits being allowed to not just take on values in $\{0, 1, 2\}$, but also slightly larger ones.

Let's say that we allow digit values to be one of $\{0, 1, 2, 3, 4\}$. Adding two such digits $0 \le x_i, y_i \le 4$ results in a value $0 \le z_i = x_i + y_i \le 8$, which we can decompose into a digit sum of $a_i = 0 \le z_i \% 3 \le 2$ and carry $c_i = 0 \le \lfloor z_i/3 \rfloor \le 2$. Now, summing the digit sum and the incoming carry, $s_i = a_i + z_{i-1} \le 4$, giving us a sum which again have all digits in $\{0, 1, 2, 3, 4\}$.

Actually turning this solution idea into code requires some care – we have to come up with systems for which instances should be responsible for which digits, how to encode the base-3 digits (e.g., using binary), and carefully time when and by whom bits are read/written. The constant factor is also not *that* much better than that of the $O(\log N \log M)$ binary adder solution; the reference solution that implements this requires 171 rounds.

**Groups of powers of two, reduced with half/full adders:** $O(\log M)$

Another (both simpler and more round-efficient) way of getting rid of the $O(\log N)$ factor from addition carry chains is to give up on keeping full binary(/ternary) numbers altogether. Instead, we keep sets of powers of two, that get successively combined and reduced into bigger powers of two until only a handful remain.

Each power of two is tied to a location, and can be either active (if the value 1 is written in the location), or inactive (if 0 is). The sum of the active powers of two is always $N$, and each active power of two has an instance that is responsible for it.

We start with $M$ copies of $2^0$ tied to locations $0, \ldots, M-1$, and have each instance write a 1 to the location give by its ID (which it is set as responsible for). Note that the sum of active powers of two is $2^0 + 2^0 + \ldots + 2^0 = N$, as expected.

In the following rounds, we pair equal powers of two up with each other, potentially with some left over. For each pair $(2^k, 2^k)$ at locations $(a, b)$, the sum of active powers of two in that pair may be either 0, $2^k$ or $2^{k+1}$. We pick out two new unused locations $(c, d)$ and assign them the values $2^k$ and $2^{k+1}$ respectively. Each instance that is responsible for an active power of two in the pair reads the other's location. If both of the powers of two are active, we have the first instance write a 1 to $d$ and take responsibility of it, while the second instance goes into a "dead" state. If only one of them are active, we have that instance write a 1 to $c$ and take responsibility of it. If none of them are active, $c$ and $d$ will both remain 0. For the next round, the sets of powers of two we consider will be those corresponding to all the new locations $(c, d)$, plus the ones that did not get paired with anything.

If we perform this process repeatedly starting with $M = 10^5$, we will see a number of powers of two that changes as following:

- $\{100000 \times 2^0\}$
- $\{50000 \times 2^0, 50000 \times 2^1\}$
- $\{25000 \times 2^0, 50000 \times 2^1, 25000 \times 2^2\}$
- $\{12500 \times 2^0, 37500 \times 2^1, 37500 \times 2^2, 12500 \times 2^3\}$
- ...

and eventually, after 37 steps (in general, $O(\log M)$ many), we will reach a stage where there is only one copy each of all of $2^0, 2^1, \ldots, 2^6$. ($2^7$ and above can be ignored, since they are bigger than $N$ and therefore never active.) At this point, all instances can read the locations corresponding to those copies and sum the powers of two that are marked as active (whose locations have a value of 1) to determine $N$.

This method uses 82 rounds: two for each step (one read + one write), plus 7 for the final reads and 1 for outputting the answer.

A slight improvement over this can be achieved by switching from combining

pairs to powers of two, to triplets. Each triplet $(2^k, 2^k, 2^k)$ can sum to either 0, $2^k$, $2^{k+1}$ or $2^k + 2^{k+1}$, meaning we can replace these three powers of two by just two new ones ($2^k$ and $2^{k+1}$). In more technical language, this means that instead of using half adders (mapping two bits $a, b$ to $a + b = 2x + y$), we use full adders (mapping three bits $a, b, c$ to $a + b + c = 2x + y$).

This method uses just 69 rounds.

There are some improvements you can make on top of this to get down to 61 rounds and get full score, although they are all rather finicky. For example, one thing you can do is circulate which nodes get to be in the dead state, and have those do productive work like eagerly reading off some powers of two and adding them to their sum. Another thing is to optimize the exact schedule of half/full adders to use towards the end of the process. The exact details here are left as an exercise to the reader.